1️⃣ Phase Wise Implementation

Here's the complete, phase-by-phase implementation breakdown for **Genie360** — no code snippets, just laser-focused functional specs for every function, notebook, and deliverable.

🚀 Genie360 — Phase-by-Phase Implementation Breakdown

⚡ PHASE 1 (MVP): SQL Query Optimization Engine

> **Goal:** Get Genie-generated SQL from `system.query.history`, detect what's broken, rewrite it, and push it back into the Genie Space.
>
> **Timeline:** Weeks 1–4

📦 Functions

Module 1: Query History Ingestion

Function	Functionality
`get_genie_queries(space_id, lookback_days)`	Queries `system.query.history` using `query_source.genie_space_id` filter. Returns raw query records including `statement_text`, `execution_duration_ms`, `read_bytes`, `read_rows`, `spilled_local_bytes`, and `execution_status`.
`normalize_query_record(raw_record)`	Flattens the nested `compute`, `query_source`, and `query_parameters` structs into a flat Python dict. Handles NULL fields gracefully.
`deduplicate_queries(query_list)`	Deduplicates queries by semantic fingerprint (normalized SQL hash). Keeps the most recent execution of structurally identical queries.
`calculate_query_cost_estimate(read_bytes, duration_ms, warehouse_type)`	Estimates USD cost based on DBU pricing for CLASSIC vs SERVERLESS warehouses. Uses `read_bytes` and `duration_ms` as proxy signals.
`enrich_query_metadata(query_record)`	Adds derived fields: cost estimate, duration tier (FAST/SLOW/CRITICAL), data volume tier (SMALL/MEDIUM/LARGE), and cache hit flag from `from_result_cache`.

Module 2: Anti-Pattern Detection

Function	Functionality
`parse_sql_to_ast(sql_text, dialect="databricks")`	Uses `sqlglot` to parse SQL into an AST (Abstract Syntax Tree). Returns structured tree for downstream rule evaluation.
`detect_select_star(ast_node)`	Traverses AST for `SELECT *` patterns. Returns flagged columns and affected table references. Severity: MEDIUM.
`detect_missing_limit(ast_node)`	Checks if the top-level SELECT has no LIMIT clause and estimated `produced_rows` exceeds threshold. Severity: HIGH for large result sets.
`detect_full_table_scan(ast_node, table_stats)`	Checks for table references with no WHERE predicates and no partition filter on partitioned Delta tables. Cross-references Unity Catalog table stats. Severity: HIGH.
`detect_correlated_subquery(ast_node)`	Identifies subqueries in WHERE/HAVING clauses that reference outer query columns — the classic N+1 problem in SQL. Severity: CRITICAL.
`detect_count_distinct_inefficiency(ast_node)`	Flags `COUNT(DISTINCT col)` expressions on large tables where `APPROX_COUNT_DISTINCT` would suffice with <2% error. Severity: MEDIUM.
`detect_cross_join_risk(ast_node)`	Detects implicit cross joins (comma-separated tables in FROM without JOIN conditions) or explicit CROSS JOINs on large tables. Severity: CRITICAL.

Function	Functionality
`detect_implicit_type_cast(ast_node, column_metadata)`	Identifies WHERE predicates that compare a string literal against an integer column or vice versa — kills partition pruning. Severity: HIGH.
`detect_repeated_subquery(ast_node)`	Detects the same subquery expression appearing more than once in the SQL — a candidate for CTE extraction. Severity: MEDIUM.
`run_anti_pattern_suite(sql_text, table_stats, column_metadata)`	Orchestrates all individual detection functions. Returns a structured `AntiPatternReport` with severity rankings, estimated impact, and fix suggestions.

Module 3: SQL Rewrite Engine

Function	Functionality
`rewrite_select_star(ast_node, column_metadata)`	Expands `SELECT *` to explicit column list using Unity Catalog column metadata. Drops system-generated `_rescue_data` columns automatically.
`inject_limit_clause(ast_node, safe_limit=10000)`	Adds a `LIMIT` clause to SELECT statements missing one. Limit value is parameterizable per space configuration.
`rewrite_subquery_to_cte(ast_node)`	Extracts repeated or correlated subqueries into WITH (CTE) blocks. Uses sqlglot's `eliminate_subqueries` optimizer as base, then applies Databricks-specific dialect fixes.
`replace_count_distinct_with_approx(ast_node, cardinality_threshold)`	Replaces `COUNT(DISTINCT col)` with `APPROX_COUNT_DISTINCT(col, 0.01)` when column cardinality exceeds configured threshold. Preserves original as a comment.
`inject_partition_filter(ast_node, partition_columns, date_range_hint)`	Adds partition predicates to queries on partitioned Delta tables when none are present. Infers likely date range from conversation context or last 30 days as safe default.
`fix_implicit_type_cast(ast_node, column_metadata)`	Wraps mismatched literals in explicit CAST expressions to preserve partition pruning.
`serialize_ast_to_sql(ast_node, dialect="databricks")`	Converts the modified AST back to a valid Databricks SQL string using sqlglot's generator.
`calculate_rewrite_impact(original_sql, rewritten_sql, query_history_stats)`	Estimates cost/performance delta by comparing estimated scan size, anti-pattern count, and historical execution patterns for similar queries.

Module 4: LLM Refinement & Validation

Function	Functionality
`build_llm_rewrite_prompt(original_sql, rule_rewritten_sql, anti_pattern_report, table_context)`	Constructs a structured prompt for Claude Sonnet that includes the original SQL, rule-based rewrite, detected issues, relevant table schemas, and Databricks-specific optimization guidelines.
`invoke_llm_rewrite(prompt, llm_endpoint)`	Calls the configured Databricks LLM serving endpoint via `mlflow.deployments.get_deploy_client`. Logs the call to MLflow tracing with input/output tokens and latency.
`parse_llm_rewrite_response(llm_response)`	Extracts the rewritten SQL, explanation text, and confidence score from the structured LLM response. Handles malformed responses with fallback to rule-based rewrite.
`compare_explain_plans(original_sql, rewritten_sql, warehouse_id)`	Runs `EXPLAIN EXTENDED` on both SQL versions against the configured SQL Warehouse. Parses output to compare estimated scan sizes, shuffle operations, and join types.
`validate_semantic_equivalence(original_sql, rewritten_sql, sample_data_limit=100)`	Runs both SQL versions on a warehouse with LIMIT 100. Compares result schema and sample row hashes to verify the rewrite produces semantically identical output.
`score_rewrite_confidence(anti_pattern_report, explain_plan_comparison, semantic_equivalence_result)`	Combines rule coverage, explain plan improvement delta, and semantic equivalence check into a single confidence score (0.0–1.0).

Module 5: Genie Space Injection

Function	Functionality
`get_genie_space(space_id, workspace_url, token)`	Calls the Genie GET API (`GET /api/2.0/genie/spaces/{id}`) to retrieve the `serialized_space` JSON — the complete space definition including instructions, example SQL, and JOIN relationships.
`deserialize_space_config(serialized_space_json)`	Parses the `serialized_space` JSON string into a Python dict for manipulation. Validates schema version and structure.
`find_matching_example_sql(space_config, original_sql)`	Searches the space's `example_question_sqls` array to find the entry matching the original query being replaced. Uses SQL hash fingerprinting for exact match, fuzzy similarity for near-matches.
`update_example_sql_entry(space_config, old_sql, new_sql, change_metadata)`	Replaces the matched example SQL entry with the optimized version. Appends change metadata (original, rewrite reason, confidence score, timestamp) as a JSON comment.
`serialize_space_config(space_config)`	Re-serializes the modified space config back to the `serialized_space` JSON string format required by the Update API.
`update_genie_space(space_id, serialized_space, workspace_url, token)`	Calls the Genie Update API (`PUT /api/2.0/genie/spaces/{id}`) with the modified serialized space. Returns success/failure status and new space version.
`batch_inject_rewrites(space_id, approved_rewrites, workspace_url, token)`	Processes a batch of approved rewrites for a single space in a single API round-trip. Minimizes API calls by bundling all changes into one Update call.
`rollback_space_update(space_id, previous_serialized_space, workspace_url, token)`	Restores a Genie Space to its previous `serialized_space` state. Called automatically on injection failure or manual rollback request.

📓 Notebooks

Notebook	Purpose
`00_setup_infrastructure.ipynb`	One-time setup: creates Lakebase PostgreSQL tables (`genie_spaces` , `query_analysis` , `rewrite_history` , `injection_log`), grants service principal permissions to `system.query.history` , configures SQL Warehouse connection.
`01_query_history_exploration.ipynb`	Exploratory analysis notebook: interactive queries against `system.query.history` filtered by `genie_space_id` , visualizes top offenders, cost distribution, anti-pattern frequency heatmap. Used during development and onboarding.
`02_anti_pattern_detection_test.ipynb`	Unit testing notebook for the detection suite: runs all 8 detection functions against a library of 20 representative Genie-generated SQL patterns. Outputs pass/fail table and AST visualizations.
`03_rewrite_pipeline_test.ipynb`	End-to-end test for the rewrite pipeline: takes 5 real queries from a connected Genie Space, runs the full rule → LLM → validation pipeline, displays side-by-side before/after with confidence scores and explain plan comparisons.
`04_grant_app_permissions.ipynb`	Permission bootstrap: grants the Databricks App's service principal CAN USE on SQL Warehouse, CAN READ on `system.query.history` , and CAN EDIT on target Genie Spaces. Adapted from GenieRx's existing grant notebook.

✅ Deliverables — Phase 1 (MVP)

#	Deliverable	Type	Description
1.1	`QueryHistoryIngester` module	Python Package	Production-grade ingestion from `system.query.history` with Genie space filtering, deduplication, cost estimation
1.2	`AntiPatternDetector` module	Python Package	8-rule detection suite using sqlglot AST, produces structured `AntiPatternReport` per query
1.3	`SQLRewriteEngine` module	Python Package	Rule-based SQL rewriter for 6 anti-pattern types, serializes clean Databricks SQL via sqlglot
1.4	`LLMRefinementOrchestrator`	Python Package	Claude Sonnet refinement + semantic equivalence validator + confidence scorer
1.5	`GenieSpaceInjector` module	Python Package	Full CRUD wrapper for Genie Space API, bulk injection + rollback capability
1.6	`00_setup_infrastructure.ipynb`	Notebook	Lakebase schema creation, permissions bootstrap

#	Deliverable	Type	Description
1.7	`01_query_history_exploration.ipynb`	Notebook	Interactive analysis dashboard for Genie query patterns
1.8	`02_anti_pattern_detection_test.ipynb`	Notebook	Validation suite for detection rules
1.9	`03_rewrite_pipeline_test.ipynb`	Notebook	E2E pipeline test with explain plan comparison
1.10	`04_grant_app_permissions.ipynb`	Notebook	Service principal permission bootstrap
1.11	Lakebase Schema v1	PostgreSQL DDL	Tables: `genie_spaces`, `query_analysis`, `rewrite_history`, `injection_log`
1.12	FastAPI `POST /api/v1/spaces/{id}/analyze`	REST Endpoint	Triggers full analysis pipeline on a space, returns `AntiPatternReport`
1.13	FastAPI `GET /api/v1/spaces/{id}/rewrites`	REST Endpoint	Returns pending rewrites for a space with confidence scores and impact estimates
1.14	FastAPI `POST /api/v1/spaces/{id}/inject`	REST Endpoint	Approves and injects a batch of rewrites into a Genie Space
1.15	FastAPI `POST /api/v1/spaces/{id}/rollback`	REST Endpoint	Restores space to pre-injection state
1.16	Basic React "Optimization Queue" UI Tab	Frontend	Table view of pending rewrites per space, before/after SQL diff, approve/reject controls

---

🔥 PHASE 2: Cost Intelligence & Alert Engine

> **Goal:** Predict query costs before execution, surface materialized view opportunities, and alert on cost anomalies.
>
> **Timeline:** Weeks 5–7

📦 Functions

Module 6: Cost Prediction Model

Function	Functionality
`extract_cost_features(query_record)`	Extracts ML features from query history records: `read_bytes`, `read_partitions`, `shuffle_read_bytes`, `spilled_local_bytes`, `total_task_duration_ms`, statement type, warehouse type, time-of-day, estimated cardinality.
`load_training_dataset(lookback_days=90)`	Pulls 90 days of query history with actual cost outcomes from Lakebase. Applies stratified sampling to balance CHEAP/MODERATE/EXPENSIVE query classes.
`train_cost_prediction_model(training_df)`	Trains an XGBoost regression model on extracted features. Logs experiment to MLflow: parameters, metrics (MAE, RMSE, R^2), feature importance chart, and model artifact.
`register_cost_model(run_id, model_name)`	Registers the trained model to MLflow Model Registry under `genie360_cost_predictor`. Sets alias `@production` on the latest passing version.
`predict_query_cost(features, model_uri)`	Loads registered model and returns predicted USD cost for a query before execution. Used in the alert middleware.
`evaluate_model_drift(current_model_uri, recent_data)`	Compares model predictions vs actual costs on last 7 days of data. Triggers retraining workflow if MAE exceeds $0.75 threshold.

Module 7: Cost Alert Middleware

Function	Functionality
`classify_cost_tier(predicted_cost)`	Maps predicted cost to alert tier: SILENT (<$1), INFO ($1–$10), CONFIRM ($10–$50), BLOCK (>$50). Returns tier enum and threshold metadata.
`format_cost_alert_message(query_text, predicted_cost, tier, rewrite_available)`	Builds human-readable alert message. Includes cost estimate, tier explanation, and link to optimized rewrite if one exists.
`send_slack_alert(channel_id, message, webhook_url)`	Posts CONFIRM/BLOCK tier alerts to configured Slack channel via incoming webhook. Includes query hash, space ID, and Databricks workspace deep-link.
`send_teams_alert(webhook_url, message)`	Microsoft Teams notification for CONFIRM/BLOCK tiers. Formatted as adaptive card with cost estimate, query snippet, and approve/bypass

Function	Functionality
	buttons.
`log_alert_event(query_id, tier, predicted_cost, actual_cost, action_taken)`	Persists alert events to Lakebase `cost_alerts` table for audit trail and model feedback loop.

Module 8: Materialized View Recommender

Function	Functionality
`extract_query_signature(sql_text)`	Normalizes SQL by stripping literals and parameters, producing a canonical query signature for clustering.
`cluster_query_signatures(query_signatures, min_frequency=5)`	Groups query signatures by structural similarity using MinHash LSH. Returns clusters with frequency count and combined scan statistics.
`estimate_mv_savings(cluster_stats, refresh_cost_per_run)`	Calculates estimated daily DBU and USD savings if a materialized view is created for a cluster. Factors in MV refresh cost vs repeated scan cost.
`generate_mv_ddl(cluster, base_table_refs, warehouse_id)`	Produces a `CREATE MATERIALIZED VIEW` DDL statement for the cluster's representative query pattern. Includes appropriate refresh schedule and `COMMENT` documenting the source query pattern.
`validate_mv_feasibility(mv_ddl, warehouse_id)`	Runs `EXPLAIN` on the MV DDL to verify it's syntactically valid and the base tables are accessible. Checks for non-deterministic functions that would block MV creation.
`rank_mv_recommendations(mv_candidates)`	Sorts MV candidates by estimated annual savings descending. Applies tie-breaking by cluster frequency and implementation complexity.

Module 9: Warehouse Right-Sizer

Function	Functionality
`analyze_warehouse_utilization(warehouse_id, lookback_days=30)`	Aggregates query history stats per warehouse: peak concurrency, P50/P90/P99 execution duration, average bytes scanned, failure rate, queue wait time.
`recommend_warehouse_size(utilization_stats)`	Applies sizing heuristics: if P90 duration < 5s and peak concurrency < 5, recommends downsize; if queue wait > 30s P50, recommends upsize. Returns recommendation with projected cost delta.
`recommend_warehouse_type(utilization_stats, query_patterns)`	Recommends CLASSIC vs SERVERLESS based on query burst patterns, idle time ratio, and concurrency profile. Serverless preferred for unpredictable workloads; Classic for sustained throughput.
`calculate_warehouse_cost_delta(current_config, recommended_config, query_volume)`	Estimates monthly savings from right-sizing based on DBU price difference and historical query volume.

📓 Notebooks

Notebook	Purpose
`05_cost_model_training.ipynb`	Full ML experiment notebook: feature engineering from query history, XGBoost training, hyperparameter tuning, MLflow experiment tracking, model registration, evaluation report with feature importance.
`06_mv_recommendation_analysis.ipynb`	Interactive MV analysis: loads 90-day query history, runs clustering, displays candidate MVs ranked by savings, outputs ready-to-execute DDL statements for review.
`07_warehouse_sizing_analysis.ipynb`	Warehouse utilization deep-dive: concurrency heatmaps per hour-of-day, P90 duration trends, cost per warehouse, sizing recommendation summary with savings projections.

✅ Deliverables — Phase 2

#	Deliverable	Type
2.1	XGBoost Cost Prediction Model	MLflow Model (Registered)
2.2	`CostAlertMiddleware`	FastAPI Middleware
2.3	`MaterializedViewRecommender` module	Python Package
2.4	`WarehouseRightSizer` module	Python Package

#	Deliverable	Type
2.5	`05_cost_model_training.ipynb`	Notebook
2.6	`06_mv_recommendation_analysis.ipynb`	Notebook
2.7	`07_warehouse_sizing_analysis.ipynb`	Notebook
2.8	`GET /api/v1/mv-recommendations/{space_id}`	REST Endpoint
2.9	`GET /api/v1/warehouse-recommendations`	REST Endpoint
2.10	`POST /api/v1/mv-recommendations/{id}/apply`	REST Endpoint
2.11	Lakebase `cost_alerts`, `mv_recommendations`, `warehouse_recommendations` tables	PostgreSQL Schema
2.12	"Cost Intelligence" React UI Tab	Frontend
2.13	"Warehouse Health" React UI Tab	Frontend

🔍 PHASE 3: Config Intelligence (GenieIQ + GenieRx)

> **Goal:** Score every Genie Space 0–100, audit configurations for best-practice compliance, and run benchmark Q&A optimization loops.
>
> **Timeline:** Weeks 8–10

📦 Functions

Module 10: Space Scoring (GenieIQ Port)

Function	Functionality
`list_all_genie_spaces(workspace_url, token)`	Calls `GET /api/2.0/genie/spaces` to retrieve all spaces in the workspace. Returns space IDs, names, owners, and last-modified timestamps.
`fetch_space_config(space_id, workspace_url, token)`	Calls `GET /api/2.0/genie/spaces/{id}` and parses the `serialized_space` field into a structured config dict.
`score_instruction_quality(space_config)`	Evaluates the `instructions` field: length adequacy (≥200 chars), specificity (avoids vague terms like "always be helpful"), domain-specific vocabulary density. Returns 0–20 score.
`score_table_coverage(space_config, unity_catalog_stats)`	Checks: number of tables added (≥3 recommended), presence of column descriptions in Unity Catalog, defined JOIN relationships. Returns 0–20 score.
`score_example_sql_quality(space_config)`	Evaluates example SQL entries: count (≥5 recommended), coverage of different query types (aggregation, filter, join, time-series), SQL validity. Returns 0–20 score.
`score_benchmark_coverage(space_id, workspace_url, token)`	Calls benchmarks API to check: benchmark question count (≥10 recommended), last run date, passing rate. Returns 0–20 score.
`score_cost_health(space_id, query_analysis_results)`	Integrates Phase 1/2 results: anti-pattern ratio, average query cost, rewrite adoption rate. Returns 0–20 score. Bridges GenieIQ and the new Cost Intelligence pillar.
`calculate_space_score(space_id)`	Orchestrates all 5 scoring dimensions. Returns composite score (0–100), maturity badge (BRONZE/SILVER/GOLD/PLATINUM), and dimension breakdown.
`batch_score_all_spaces(workspace_url, token)`	Runs `calculate_space_score` for every space in the workspace. Writes results to Lakebase `space_scores` table with delta tracking vs previous score.

Module 11: Config Auditor (GenieRx Port)

Function	Functionality
`run_instruction_audit(space_config)`	Checks instruction section against 12 best-practice rules: presence of domain context, date format guidance, metric definitions, handling of edge cases, avoidance of ambiguous terms.
`run_table_config_audit(space_config, unity_catalog_client)`	Verifies: all referenced tables exist and are accessible, column descriptions present for key columns, no deprecated table references, primary/foreign keys or JOIN relationships defined.
`run_sql_examples_audit(space_config)`	Validates all example SQL entries: syntax validity, dialect compatibility, reasonable complexity distribution, no hardcoded dates/user IDs.

Function	Functionality
`build_audit_llm_prompt(space_config, rule_audit_results)`	Constructs a structured prompt for Claude Sonnet with the full space configuration and rule-based audit findings. Requests narrative recommendations per section.
`invoke_audit_llm(prompt, llm_endpoint)`	Calls Claude Sonnet for audit narrative generation. Logs to MLflow with input/output tokens. Returns structured JSON with section scores and improvement recommendations.
`generate_audit_report(space_id, rule_results, llm_narrative)`	Compiles complete `AuditReport` object: section-level scores, compliance checklist, priority-ranked improvement actions, and estimated impact of each fix.

Module 12: Benchmark Q&A Optimizer (GenieRx Port)

Function	Functionality
`fetch_benchmark_questions(space_id, workspace_url, token)`	Retrieves all benchmark questions from a Genie Space via API. Returns question text, gold-standard SQL, and last evaluation result.
`run_genie_benchmark_question(space_id, question, workspace_url, token)`	Starts a Genie conversation via `POST /api/2.0/genie/spaces/{id}/conversations` and sends the benchmark question. Polls for completion. Returns generated SQL and answer.
`compare_benchmark_result(gold_sql, generated_sql, warehouse_id)`	Executes both SQL variants and compares: result schema match, row count match, sample value similarity. Returns PASS/FAIL with detailed diff.
`label_benchmark_feedback(space_id, message_id, feedback, workspace_url, token)`	Calls the feedback API (`POST /api/2.0/genie/spaces/{id}/conversations/{cid}/messages/{mid}/feedback` to submit thumbs up/down.
`generate_optimization_suggestions(failing_benchmarks, space_config)`	Analyzes patterns in failing benchmarks: common table misses, ambiguous metric names, missing JOIN paths. Generates prioritized instruction improvements.
`create_optimized_space(source_space_id, suggestions, workspace_url, token)`	Creates a new Genie Space via `POST /api/2.0/genie/spaces` with the optimized configuration. Stores old vs new `serialized_space` JSON diff in Lakebase.
`generate_space_diff(old_config, new_config)`	Produces a structured JSON diff showing exactly what changed between two space configurations. Used for the side-by-side UI preview.

📓 Notebooks

Notebook	Purpose
`08_space_scoring_batch.ipynb`	Runs batch scoring across all spaces, produces scoring leaderboard with trend lines, identifies spaces needing immediate attention (score < 40).
`09_config_audit_deep_dive.ipynb`	Interactive audit notebook: select a space, run full audit suite, display section-by-section checklist with LLM narrative, generate fix-it action plan.
`10_benchmark_qa_loop.ipynb`	Benchmark optimization loop: select a space, run all benchmark questions, display pass/fail matrix, label failures, generate instruction improvements, preview optimized space diff.

✅ Deliverables — Phase 3

#	Deliverable	Type
3.1	`GenieSpaceScorer` module	Python Package
3.2	`GenieSpaceAuditor` module	Python Package
3.3	`BenchmarkQAOptimizer` module	Python Package
3.4	`08_space_scoring_batch.ipynb`	Notebook
3.5	`09_config_audit_deep_dive.ipynb`	Notebook
3.6	`10_benchmark_qa_loop.ipynb`	Notebook
3.7	`GET /api/v1/spaces`	REST Endpoint (with scores)
3.8	`GET /api/v1/spaces/{id}/score`	REST Endpoint
3.9	`POST /api/v1/spaces/{id}/audit`	REST Endpoint
3.10	`POST /api/v1/spaces/{id}/optimize`	REST Endpoint

#	Deliverable	Type
3.11	`GET /api/v1/spaces/{id}/benchmark-results`	REST Endpoint
3.12	Lakebase `space_scores`, `audit_results`, `benchmark_results` tables	PostgreSQL Schema
3.13	"Space Catalog" React UI Tab	Frontend (with scores + maturity badges)
3.14	"Config Audit" React UI Tab	Frontend (compliance checklist + LLM narrative)
3.15	"Benchmark Q&A" React UI Tab	Frontend (benchmark matrix + side-by-side diff)

🔄 PHASE 4: Continuous Learning Loop

> **Goal:** Automate the entire pipeline nightly — ingest, detect, rewrite, inject, rescore — with feedback-driven model improvement.
>
> **Timeline:** Weeks 11–12

📦 Functions

Module 13: Auto-Improve Pipeline

Function	Functionality
`get_auto_inject_eligible_rewrites()`	Queries Lakebase for rewrites with `confidence_score >= 0.90` that have not yet been injected and have been pending for ≤ 24 hours.
`apply_injection_with_circuit_breaker(space_id, rewrites)`	Wraps `batch_inject_rewrites` with a circuit breaker: if 3 consecutive injections for the same space fail, disables auto-inject for that space and triggers a Slack alert.
`collect_post_injection_metrics(space_id, injected_queries, lookback_hours=24)`	After injection, monitors query history for the rewritten queries. Compares pre/post: execution duration, cost estimate, anti-pattern count.
`calculate_roi_metrics(space_id, post_injection_metrics)`	Computes actual dollar savings vs predicted savings. Updates `injection_log` with realized ROI. Feeds actuals back to cost prediction model training dataset.
`update_model_training_data(new_actuals)`	Appends realized cost outcomes to the model training dataset in Delta Lake. Triggers drift evaluation via `evaluate_model_drift`.
`trigger_retraining_if_needed(drift_evaluation_result)`	Dispatches a Databricks Workflow job run for `05_cost_model_training.ipynb` if drift evaluation detects MAE > threshold.

Module 14: Nightly Workflow Orchestration

Function	Functionality
`build_nightly_workflow_config(workspace_url, space_ids)`	Programmatically generates the Databricks Workflow JSON for the nightly improvement pipeline. Tasks: ingest → detect → rewrite → validate → auto-inject → rescore → report.
`dispatch_nightly_run(workflow_id, token)`	Triggers the nightly workflow run via Jobs API. Returns `run_id` for monitoring.
`monitor_workflow_run(run_id, token)`	Polls Jobs API for run completion. Returns per-task status, duration, and error messages if any task fails.
`generate_nightly_digest(run_summary, space_scores_delta)`	Produces a structured email digest: spaces improved, rewrites injected, cost savings realized, model drift status, any failures requiring human review.
`send_digest_email(digest, recipient_list)`	Dispatches the nightly digest via Databricks notification destinations (supports email, Slack, Teams).

📓 Notebooks

Notebook	Purpose
`11_auto_improve_loop_test.ipynb`	End-to-end test of the auto-improvement loop on a test Genie Space: runs all 7 pipeline stages, verifies each stage's output, validates rollback behavior on simulated failure.
`12_roi_tracking_dashboard.ipynb`	Aggregates realized savings across all spaces: before/after cost comparison charts, cumulative savings trend, space-level ROI leaderboard, model accuracy tracking over time.

✅ Deliverables — Phase 4

#	Deliverable	Type
4.1	`AutoImprovePipeline` module	Python Package
4.2	`NightlyWorkflowOrchestrator` module	Python Package
4.3	Databricks Workflow YAML (nightly pipeline)	Infrastructure as Code
4.4	`11_auto_improve_loop_test.ipynb`	Notebook
4.5	`12_roi_tracking_dashboard.ipynb`	Notebook
4.6	`GET /api/v1/pipeline/status`	REST Endpoint
4.7	`POST /api/v1/pipeline/trigger`	REST Endpoint
4.8	Lakebase `pipeline_runs`, `roi_metrics` tables	PostgreSQL Schema
4.9	Nightly digest email/Slack notification	Operational Output

🌐 PHASE 5: Unified Platform & Enterprise Features

> **Goal:** Unify all pillars in a single React admin dashboard, add multi-workspace support, governance controls, and deploy as a production Databricks App.
>
> **Timeline:** Weeks 13–14

📦 Functions

Module 15: Multi-Workspace & Governance

Function	Functionality
`register_workspace(workspace_url, token, workspace_name)`	Adds a new workspace to Genie360's managed scope. Validates connectivity, runs permission check, stores credentials in Databricks Secrets.
`aggregate_cross_workspace_metrics()`	Consolidates space scores, query analysis, and ROI metrics across all registered workspaces. Produces unified org-level view.
`enforce_injection_governance_policy(space_id, rewrite)`	Checks configured governance rules before any injection: auto-inject disabled for PRODUCTION-tagged spaces, requires human approval for BUSINESS_CRITICAL spaces, logs all decisions.
`apply_workspace_filter(user_identity, spaces_list)`	Filters visible spaces based on Unity Catalog permissions. Genie360 only shows spaces the requesting user has access to, never elevating permissions.

Module 16: Unified Dashboard API

Function	Functionality
`get_dashboard_summary()`	Aggregates key metrics for the home dashboard: total spaces managed, org-wide health score, total realized savings to date, active rewrites pending approval, spaces below score threshold.
`get_roi_summary(time_range)`	Returns ROI metrics for the selected time range: rewrites applied, cost savings by pillar (SQL opt, MV, warehouse sizing), improvement trend chart data.
`export_audit_report(space_id, format)`	Generates a downloadable audit report for a space in PDF or JSON format. Includes score history, compliance checklist, benchmark results, and rewrite log.

📓 Notebooks

Notebook	Purpose
`13_production_deployment.ipynb`	Final deployment checklist: runs all permission grants, validates all API endpoints, deploys Databricks App, runs smoke tests against production Genie Spaces, confirms nightly workflow schedule.
`14_monitoring_and_alerting_setup.ipynb`	Sets up Databricks lakehouse monitoring on all Lakebase tables, configures quality monitors for the cost model, creates Genie360 health-check dashboard in AI/BI.

✅ Deliverables — Phase 5

#	Deliverable	Type
5.1	`MultiWorkspaceManager` module	Python Package
5.2	`GovernancePolicyEngine` module	Python Package
5.3	`13_production_deployment.ipynb`	Notebook
5.4	`14_monitoring_and_alerting_setup.ipynb`	Notebook
5.5	Unified React Admin Dashboard (all tabs)	Frontend Application
5.6	ROI Dashboard Tab	React Component
5.7	Improvement History Tab	React Component
5.8	Org-wide Top Offenders Tab	React Component
5.9	Production Databricks App deployment	Deployed App
5.10	`app.yaml` with full environment config	Configuration
5.11	`README.md` + Architecture Docs	Documentation
5.12	Lakehouse Quality Monitors (cost model + tables)	MLflow / DQM

📊 Complete Deliverables Summary

Phase	Functions	Notebooks	API Endpoints	UI Tabs	Timeline
Phase 1 — MVP SQL Optimization	30	5	5	1	Weeks 1–4
Phase 2 — Cost Intelligence	16	3	4	2	Weeks 5–7
Phase 3 — Config Intelligence	19	3	6	3	Weeks 8–10
Phase 4 — Continuous Learning	10	2	2	0	Weeks 11–12
Phase 5 — Unified Platform	8	2	1	3	Weeks 13–14
TOTAL	**83**	**15**	**18**	**9**	**14 Weeks**

🔑 Key Technical References

All implementation must align with:

- **Genie Space API Reference** — Create/Update/Get APIs (Beta as of Nov 2025) with `serialized_space` JSON
- **system.query.history schema** — `query_source.genie_space_id` is the key filter field
- **Azure Databricks Genie Docs** — Space configuration best practices
- **GenieRx Reference Implementation** — Audit + optimize patterns to port
- **databrickslabs** — Production-grade Databricks tooling patterns
- **databricks-industry-solutions** — Reference architectures for app deployment
- **sqlglot** `eliminate_subqueries` — Base optimizer for CTE extraction